

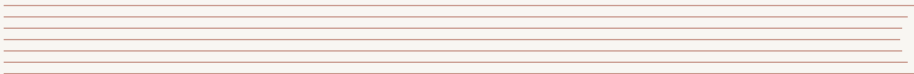
BLOOM FILTERS

A PROBABILISTIC DATA STRUCTURE FOR FAST, SPACE-EFFICIENT SET-MEMBERSHIP TESTING

AFIF SIDDIQUE (2305123) MOUMITA SARKAR MUNNY (2305136)
TAFSIR AL NAFIN (2305138)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY (BUET)
SUPERVISED BY ANIK SAHA (AKS)

SEPTEMBER 8, 2026



bloom
filter



CHECKING 3 BILLION USERNAMES INSTANTLY

When you pick a Facebook username, it must be checked for uniqueness against **every account that already exists** — and the answer has to come back as you type.

- Facebook has over **3 billion** accounts
- The check must run in **milliseconds**, on every keystroke
- It runs on **every server** that handles sign-ups

Naive Approach

Keep every username in a hash set:

~**150 GB**

far too large to hold in RAM

With a Bloom Filter

Same 3 billion usernames:

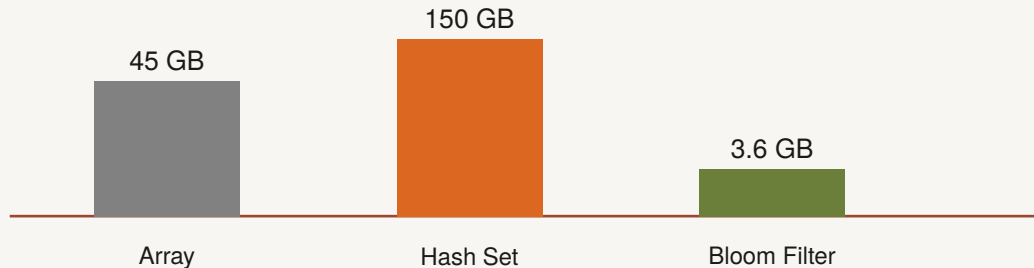
~**3.6 GB**

at a 1% false-positive rate

WHERE ARRAYS AND HASH SETS FAIL

This isn't just a Facebook problem — the same bottleneck appears in:

- **Databases** (Cassandra, HBase) — skip disk reads for missing keys
- **Browsers** (Chrome) — check URLs against malware blocklists
- **CDNs & caches** — avoid fetching uncached content
- **Blockchain** — lightweight clients checking transactions



Memory to index 3 billion usernames at 1% FPR — a **40×** reduction (*scale compressed*).

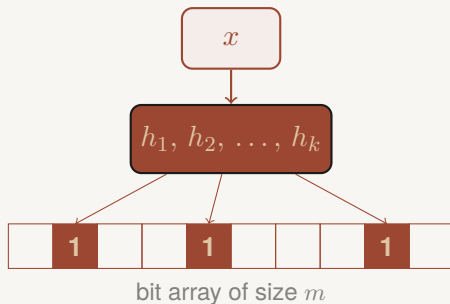
WHAT IS A BLOOM FILTER?

A Bloom filter is a **probabilistic** data structure that tests whether an element is a member of a set — using far less memory than storing the set itself.

- A bit array of size m (all bits start at 0)
- k independent hash functions
- Two possible answers only

The Core Trade-off

No false negatives — but **false positives** are possible, at a small, tunable rate.



TWO OPERATIONS: INSERT AND QUERY

A Bloom filter is a bit array $B[0 \dots m - 1]$ with k hash functions. Both operations touch exactly k bits, so both run in $O(k)$ time — *independent of n* , the number of items stored.

```
1 BLOOM-INSERT( $B, x$ )
2   for  $i \leftarrow 1$  to  $k$  do
3      $j \leftarrow \text{hash}_i(x) \bmod m$ 
4      $B[j] \leftarrow 1$ 
```

```
1 BLOOM-QUERY( $B, x$ )
2   for  $i \leftarrow 1$  to  $k$  do
3      $j \leftarrow \text{hash}_i(x) \bmod m$ 
4     if  $B[j] = 0$  then
5       return ABSENT
6   return PROBABLY-PRESENT
```

A university library holds **5 million** catalogue IDs. Here is a toy filter with $m = 16$ and $k = 3$, starting empty:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

 i *j* line 4

WALKTHROUGH: REGISTERING BK-1042

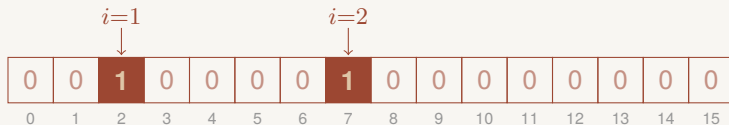
A university library holds **5 million** catalogue IDs. Here is a toy filter with $m = 16$ and $k = 3$, starting empty:



i	j	line 4
1	2	$B[2] \leftarrow 1$

WALKTHROUGH: REGISTERING BK-1042

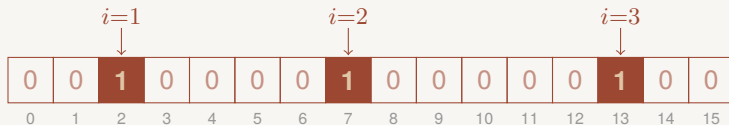
A university library holds **5 million** catalogue IDs. Here is a toy filter with $m = 16$ and $k = 3$, starting empty:



i	j	line 4
1	2	$B[2] \leftarrow 1$
2	7	$B[7] \leftarrow 1$

WALKTHROUGH: REGISTERING BK-1042

A university library holds **5 million** catalogue IDs. Here is a toy filter with $m = 16$ and $k = 3$, starting empty:



i	j	line 4
1	2	$B[2] \leftarrow 1$
2	7	$B[7] \leftarrow 1$
3	13	$B[13] \leftarrow 1$

One pass of the loop

Three iterations, three bits set. The ID BK-1042 itself is **never stored**.

WALKTHROUGH: A SECOND BOOK, SHARED BITS

A second book is catalogued. Bits from BK-1042 stay set (shown faded); the new book's bits are solid:



i j line 4

WALKTHROUGH: A SECOND BOOK, SHARED BITS

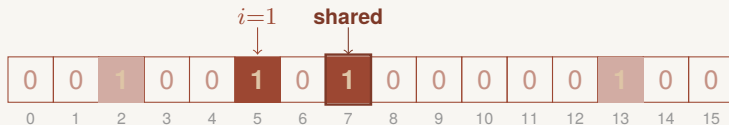
A second book is catalogued. Bits from BK-1042 stay set (shown faded); the new book's bits are solid:



i	j	line 4
1	5	$B[5] \leftarrow 1$

WALKTHROUGH: A SECOND BOOK, SHARED BITS

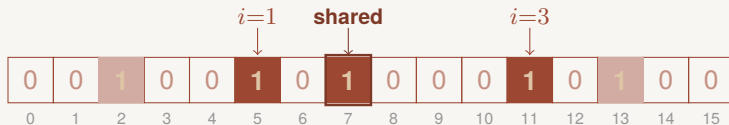
A second book is catalogued. Bits from BK-1042 stay set (shown faded); the new book's bits are solid:



<i>i</i>	<i>j</i>	line 4
1	5	$B[5] \leftarrow 1$
2	7	already 1

WALKTHROUGH: A SECOND BOOK, SHARED BITS

A second book is catalogued. Bits from BK-1042 stay set (shown faded); the new book's bits are solid:



i	j	line 4
1	5	$B[5] \leftarrow 1$
2	7	already 1
3	11	$B[11] \leftarrow 1$

Bit 7 now belongs to both

Nothing records *which* book set it. This shared ownership is the root of every false positive.

WALKTHROUGH: A CLEAN MISS

Is this book in the collection? Probe the three positions:

0	0	1	0	0	1	0	1	0	0	0	1	0	1	0	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

i j line 4

WALKTHROUGH: A CLEAN MISS

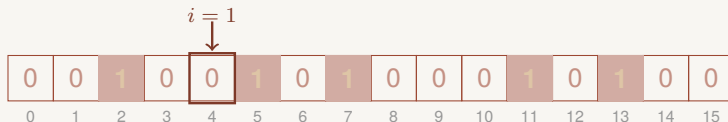
Is this book in the collection? Probe the three positions:



i	j	line 4
1	4	$B[4] = 0$

WALKTHROUGH: A CLEAN MISS

Is this book in the collection? Probe the three positions:



i	j	line 4
1	4	$B[4] = 0$
2	—	<i>not reached</i>
3	—	<i>not reached</i>

Returns ABSENT — guaranteed

One zero is enough. Line 5 returns immediately, after a **single probe**. This answer is never wrong.

WALKTHROUGH: A FALSE ALARM

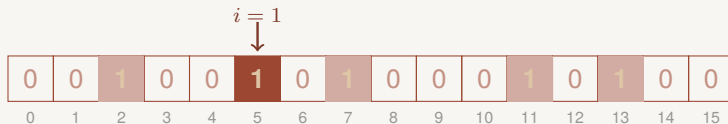
A book that was **never catalogued**. All three probes land on bits set by *other* books:



i j set by

WALKTHROUGH: A FALSE ALARM

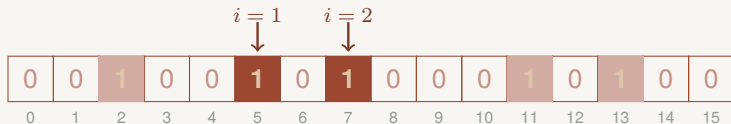
A book that was **never catalogued**. All three probes land on bits set by *other* books:



i	j	set by
1	5	BK-2087

WALKTHROUGH: A FALSE ALARM

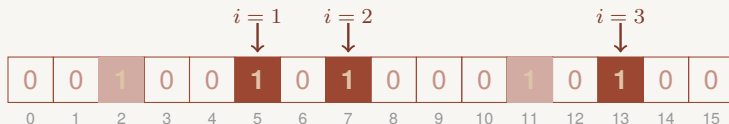
A book that was **never catalogued**. All three probes land on bits set by *other* books:



i	j	set by
1	5	BK-2087
2	7	both books

WALKTHROUGH: A FALSE ALARM

A book that was **never catalogued**. All three probes land on bits set by *other* books:



i	j	set by
1	5	BK-2087
2	7	both books
3	13	BK-1042

Returns **PROBABLY-PRESENT**

A false positive. No single book set all three bits — they were set *collectively*. The library resolves it with one exact catalogue lookup.

WHY DO FALSE POSITIVES HAPPEN?

Every insert flips k random bits to 1. The more items we add, the **fuller** the array gets. One hash misses a given bit with probability $1 - \frac{1}{m}$. After n inserts (k hashes each), that bit is *still* 0 with probability $(1 - \frac{1}{m})^{kn} \approx e^{-kn/m}$ for large m .

$$\rho \approx 1 - e^{-kn/m} \quad (\text{fraction of bits that are 1})$$

A query is a false positive when all k probed bits are **already** 1 — just by chance:

$$p \approx \rho^k$$

TUNING k AND m FOR BEST ACCURACY

- **Too few** hash functions — the array stays empty, wasting bits
- **Too many** — it fills up fast, and everything starts looking “present”

Minimizing p over k (for fixed m, n) gives:

$$k^* = \frac{m}{n} \ln 2 \implies \rho = \frac{1}{2}, \quad p_{\min} = 2^{-k}$$

a well-tuned filter is always exactly half full — the $\rho = 1/2$ from before

To hit a target rate p , substitute $k = k^*$ into $p_{\min} = 2^{-k}$ and solve for m :

$$\begin{aligned} p &= 2^{-(m/n) \ln 2} \implies \ln p = -\frac{m}{n} (\ln 2)^2 \\ \implies m &= -\frac{n \ln p}{(\ln 2)^2} \approx 1.44 n \log_2 \frac{1}{p} \end{aligned}$$

At $p = 1\%$: $k \approx 7$, $m/n \approx 9.6$ bits/element

THE PRACTICAL RECIPE

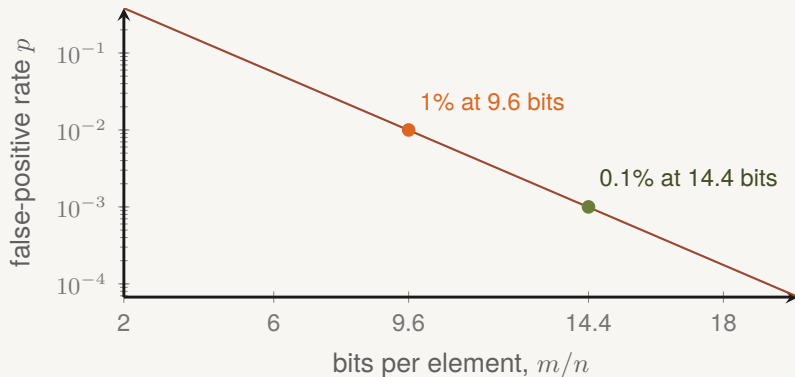
Given n items and a target error rate p

- 1 Decide the false-positive rate you can live with, p
- 2 Size the array: $m \approx 1.44 n \log_2 \frac{1}{p}$
- 3 Pick the hash count: $k \approx \frac{m}{n} \ln 2$

target p	bits/element	k
10%	4.8	3
1%	9.6	7
0.1%	14.4	10
0.01%	19.2	13

Tighter accuracy always costs more bits — there's no free lunch, only a **knob you control**.

HOW MANY BITS DO WE ACTUALLY NEED?



Tightening the target rate $10\times$ costs about 4.8 more bits/element ($9.6 \rightarrow 14.4$) — this is the curve behind the Facebook numbers: $9.6 \text{ bits} \times 3 \text{ billion accounts} \approx 3.6 \text{ GB}$.

HOW FAST ARE INSERT AND QUERY?

- INSERT — exactly k probes $\Theta(k)$
- QUERY — early exit on any 0 $\Omega(1) \dots O(k)$

INSERT can never exit early — every one of the k bits must be written. Only QUERY benefits from stopping the moment it sees a 0.

Expected probes on a miss, with ρ = fraction of bits set:

$$\mathbb{E}[N] = \sum_{i=0}^{k-1} \rho^i = \frac{1 - \rho^k}{1 - \rho} \xrightarrow{\rho=1/2} < 2$$

- A tuned filter sets exactly **half** its bits — so $\rho = 1/2$
- Under **two probes** on average, however large k grows

k fixed at design time $\implies O(1)$ in n

WHY THIS ACTUALLY HELPS

A cheap guard in front of an expensive lookup.

$$\mathbb{E}[c] = c_{\text{bf}} + p \cdot c_{\text{db}}$$

Bloom probe (RAM)	1 μs
Database lookup	10 ms
False-positive rate	1%

- 99% of non-members are rejected in RAM, in microseconds
- The win is **avoided work**, not raw speed

10 ms $\longrightarrow$ 0.1 ms $\sim$ **100 $\times$ fewer database hits**

This pattern is often called a *negative cache* in systems design.

WHERE BLOOM FILTERS ARE USED TODAY



Databases

Cassandra, HBase —
skip ~ 10 ms disk reads
for missing keys



Web & CDNs

Chrome, Akamai — filter
malicious URLs across
billions of checks/day



Bitcoin

SPV wallets — filter
transactions without the
full chain (BIP 37)



Spell Checkers

Bloom's original 1970
use case [1] — *is this
word in the dictionary?*

PROBLEM: NO DELETION

The Problem

Once a bit is set, it can never be safely cleared. Bits are **shared** between elements — clearing one could erase another element's presence too.

The Fix: Counting Bloom Filter

Replace each bit with a small **counter**. Insert increments it, delete decrements it — a count of zero means truly absent. [2]

PROBLEM: FIXED CAPACITY

The Problem

A filter is sized for a target n . Exceed it, and the false-positive rate silently climbs — there's no way to grow the array without rebuilding from scratch.

The Fix: Scalable Bloom Filter

Chain a sequence of filters together. When the current one fills up, add a new, larger one — a query checks every filter in the chain.

PROBLEM: SPACE OVERHEAD

The Problem

Classical Bloom filters need noticeably more space than the theoretical minimum to hit very low false-positive rates — and still can't support deletion.

The Fix: Cuckoo Filter

Store short **fingerprints** in a cuckoo hash table instead of raw bits. Matches or beats Bloom filter space efficiency — and deletion comes for free. [3]

PROBLEM: ONE-SIZE-FITS-ALL HASHING

The Problem

A classical filter treats every possible key as equally likely — even when real-world keys follow patterns a computer could learn.

The Fix: Learned Bloom Filter

Train a small **model** to handle most queries directly; fall back to a backup filter only for its mistakes. [4]

- [1] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [2] Li Fan, Pei Cao, Jussara Almeida, and Andrei Z. Broder. Summary cache: A scalable wide-area web cache sharing protocol. *IEEE/ACM Transactions on Networking*, 8(3):281–293, 2000.
- [3] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 75–88, 2014.
- [4] Michael Mitzenmacher. A model for learned Bloom filters and optimizing by sandwiching. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

THE BIG PICTURE

- A Bloom filter answers “**probably in**” or “**definitely not**” — using bits, not the items themselves
- **No false negatives**, tunable false positives — that trade-off is what buys the space savings
- $O(k)$ time, $O(1)$ in n — constant work, no matter how large the set grows
- A **cheap guard** in front of an expensive lookup — not a replacement, an accelerant

Thank You